

GrowFundCurv1D: Computation of one-dimensional manifolds in maps with MATLAB v3.0.0

Dana C'Julio, Bernd Krauskopf and Hinke M. Osinga



1 About `GrowFundCurv1D`

The package `GrowFundCurv1D` is a MATLAB implementation of the computational tool described in [2]. It is designed for the numerical computation of one-dimensional (un)stable manifolds of a fixed point in three-dimensional maps, identifying their intersection points with a section, and visualising the results. Beginning with version 3, the package includes the capability to store the indices of approximate preimages for all mesh points, enabling the reconstruction of pseudo-orbits; for further details regarding this feature, we refer the reader to [1]. In case you use the package `GrowFundCurv1D` in your own work, we ask that you cite [2] as its source. If you use the capability of the algorithm to extract a pseudo orbit, please cite [1].

The `GrowFundCurv1D` package is available for download at:

`github.com/dcjulio/Computing-1D-manifolds-in-maps`

This repository contains the manual `GrowFundCurv1D_manual_v3.0.0` and the folder `GrowFundCurv1D_code`, which includes:

<code>GrowFundCurv1D_demo.m</code> :	a demo script (tested on MATLAB R2025b) that computes and plots the unstable manifold $W^u(p^+)$ of the Hénon-like map H , as illustrated in Fig. 1.
<code>GrowFundCurv1D_functions</code> :	a folder containing all required MATLAB function files.

The algorithm begins by computing an initial segment of the manifold via linear interpolation in the direction of the eigenvector associated with the fixed point, using a small step δ . Starting from this segment, the algorithm identifies a fundamental domain and initiates the iterative growth process. During this process, the algorithm stores the approximate preimages of each new mesh point to facilitate pseudo-orbit construction. As a post-processing step, the intersection points of the computed manifold with a pre-specified plane are determined as an ordered set. Our research shows that, with appropriate accuracy settings, the algorithm accurately computes manifolds for our test map up to its fourth iterations without losing resolution. For a comprehensive explanation of the algorithm's performance and accuracy constraints, see [2]. For documentation on pseudo-orbit computation and its application to boundary value problems (BVP) and parameter continuation, please refer to [1].

This manual is organised as follows. In Sec. 2, we introduce the three-dimensional Hénon-like map used throughout this document and demo. Sec. 3 provides a walk-through of the demo script `GrowFundCurv1D_demo.m`, which computes and plots the unstable manifold of a fixed point, determines its intersection points with a plane, and extracts a pseudo-orbit. In Sec. 4, we provide an outline of the structure of the algorithm, which is organised into four primary steps. These steps are explained in detail in the subsequent sections: Sec. 5 covers the initialisation of the system; Sec. 6

describes the configuration of options and parameters; Sec. 7 explains the main routine and the resulting data structure; and Sec. 8 details the post-processing functions.

2 The three dimensional Henon-like map

Our example map used in the demo is the Hénon-like family of diffeomorphisms $H : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ defined by

$$H(x, y, z) = (y, \alpha - y^2 - \beta x, \xi z + y) \quad (1)$$

has been introduced and studied as the first example of an explicitly given family of diffeomorphisms that has a *blender* [? ? ?]. To be specific, we consider the parameter choice

$$\alpha = 4.2, \quad \text{and} \quad \beta = -0.3, \quad \text{and} \quad \xi = 0.8 \quad (2)$$

for which the underlying Hénon map is known to have a full horseshoe [?]. The map H has two fixed points

$$p^\pm := \left(\rho^\pm, \rho^\pm, \frac{\rho^\pm}{1 - \xi} \right), \quad (3)$$

where $\rho^\pm = \frac{1}{2} \left(-1 - \beta \pm \sqrt{4\alpha + (1 + \beta)^2} \right)$, which (certainly) exist for all $\alpha \geq 0$. For $\alpha = 4.2$, $\beta = -0.3$, and any $|\xi| < 1$, both fixed points $p^\pm$ have one-dimensional unstable manifolds $W^u(p^\pm)$ and two-dimensional stable manifolds $W^s(p^\pm)$. Figure ?? shows the manifolds $W^u(p^+)$ in a gradient or red for the parameter choice in (2). The fixed points $p^\pm$ are shown as golden dots. The data for this figure has been computed with the algorithm `GrowFundCurv1D` and the plot has being made by using the function `manifplot` as detailed in Sec. 8.2.

We compute the intersection set of $W^u(p^+)$ with the half-plane

$$\Sigma = \{(x, y, z) \in \mathbb{R}^3 \mid x = y, x > 0\},$$

which contains p^+ . While weaving through Σ , the manifold $W^u(p^+)$ makes longer and longer excursions near infinity. Therefore, we follow [?] and compactify (x, y, z) -space by the transformation

$$\mathcal{T}(x, y, z) = (\bar{x}, \bar{y}, \bar{z}) := \left(\frac{(x, y)}{1 + \sqrt{1 + (x^2 + y^2)}}, \frac{z}{1 + \sqrt{1 + z^2}} \right)$$

to the cylinder

$$\mathcal{C} = \mathcal{D} \times \mathcal{I} = \{(\bar{x}, \bar{y}, \bar{z}) \in \mathbb{R}^3 \mid \|(\bar{x}, \bar{y})\| \leq 1 \text{ and } |\bar{z}| \leq 1\}.$$

Its boundary $\partial\mathcal{C}$ represents infinity of (x, y, z) -space, and H acts on the interior of $\mathcal{C}$ as the conjugate $\mathcal{T} \circ H \circ \mathcal{T}^{-1}$. For simplicity, we refer to this conjugate map as H as well, and also use the same labels for its fixed points and invariant manifolds. Crucially, excursions of $W^u(p^+)$ near infinity are bounded in arclength in $\mathcal{C}$; this important property of the compactification makes it possible to find hundreds of its intersection points with Σ .

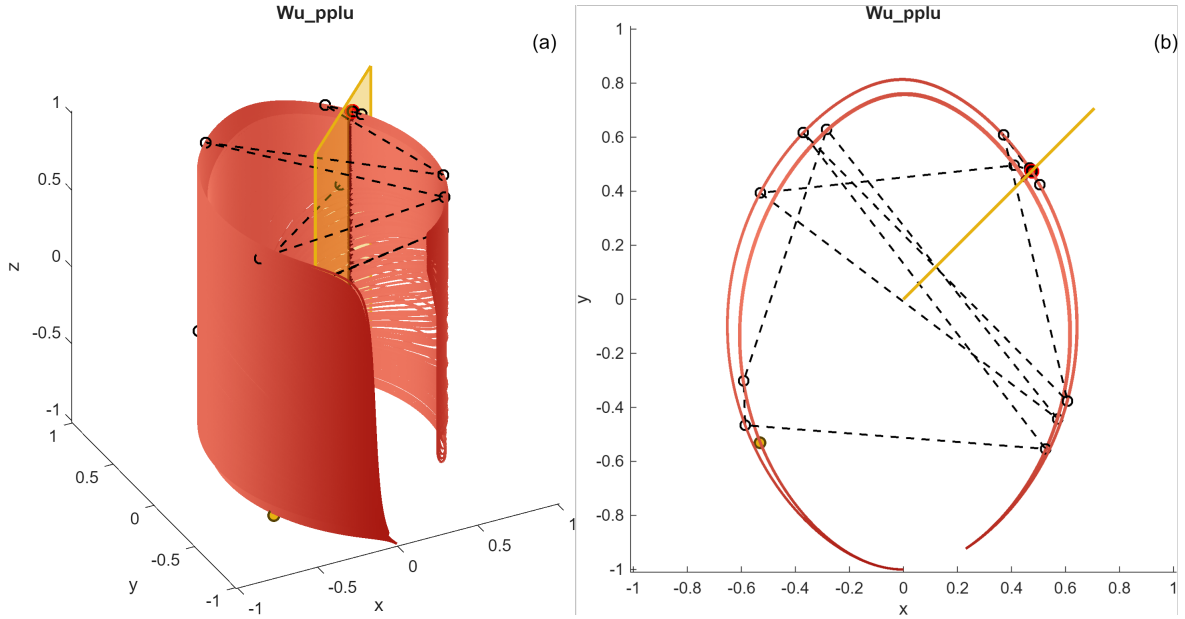


Figure 1: Output of the function `manifplot` for the diffeomorphism H with parameters $\alpha = 4.2$, $\beta = -0.3$, and $\xi = 0.8$ in compactified (x, y, z) -space. Shown are the two fixed points $p^\pm$ (golden dots) and the one-dimensional unstable manifold $W^u(p^+)$ (red curve) intersecting the plane Σ (yellow plane). Also shown is a pseudo-orbit (black circles connected by dashed lines). Panel (a) shows the three-dimensional view and panel (b) is the projection onto the (x, y) -plane.

3 Outline of the Demo

We include the demo script `GrowFundCurv1D_demo.m` with the package `GrowFundCurv1D`, which computes and plots the invariant manifold of a fixed point. Specifically, the map is the three-dimensional Hénon-like map H , the parameters are $\alpha = 4.2$, $\beta = -0.3$ and $\xi = 0.8$, and we compute both branches of the orientation-reversing unstable manifold $W^u(p^+)$ associated with the fixed point p^+ . Then it finds the intersection points of $W^u(p^+)$ with the plane Σ and the pseudo-orbit ending near the last intersection point of the manifold with Σ ; see Fig. 1.

To run the demo, ensure all files and folders are in the search path of the active MATLAB directory. The demo is then run by typing `GrowFundCurv1D_demo` into the command window of MATLAB. The following tasks are then performed:

- the paths to the necessary functions are added;
- the map information is specified and stored in the structure `opts`;
- `GrowFundCurv1D` is called to compute the manifold, here we define extra options in `opts` for the correct manifold computation;
- the function `inter_plane` is called to find the intersection points with the pre-specified

plane Σ ;

- the function `manifplot` is called to plot the manifold, the plane and its intersection points; and
- the `pseudo_orbit` function is called to extract and store the pseudo-orbit terminating near the final intersection point of the manifold with the plane. This orbit is then overlaid onto the figure generated by `manifplot`. The final results are illustrated in Fig. 1.

This demo is implemented with the default accuracy parameters referenced in Sec. 6.

4 Structure of the algorithm

The algorithm is organised into the following four steps:

- Step 1:** In this initial step, the user defines the system and its fixed points. This is achieved by creating a class of methods. This class is referred to as `StdHenon3D` in this guide and demo. Here, the user can also specify any phase space compactification. A detailed explanation of this class structure is provided in Sec. 5.
- Step 2:** In this step, the user provides parameter values, accuracy parameters, the arclength of the manifold and maximum number of iterations. This information is organised in a structure named `opts`, explained in Sec. 6. The structure `opts` is passed as an argument to the main routine `GrowFundCurv1D`, which computes the manifold and writes a comprehensive data structure called `manif`. This structure stores all configuration specifications and is subsequently updated in Step 3 with the manifold computation. Details of the `manif` data structure are in Sec. 7.1.
- Step 3:** This step is the core implementation of the algorithm from [2], and is described in Sec. 7. The last update (v3.0) stores the approximate pre-image for each mesh point to construct a pseudo-orbits, explained in [1]. The process begins by calling the routine `GrowFundCurv1D`, which defines an initial fundamental domain in a pre-specified distance δ of the fixed point in the direction of its local eigenvector [1]. After finding the first fundamental domain, it iterates it until producing the manifold up to a user-specified arclength or the maximum iteration count. After calling the main routine `GrowFundCurv1D`, users can call the function `add_branch`, which computes and adds another branch of the manifolds to the structure `manif`. The output of `GrowFundCurv1D` is the structure `manif` and is described in Sec. 7.1.

Step 4: This is a post processing step, with three functions serving distinct purposes. (1) The function `inter_plane`; computes the ordered intersection points of the manifold with a pre-specified plane, it stores them in the structure `manif`; see Sec. 8.1. (2) The function `pseudo_orbit` reconstructs pseudo-orbits starting from any mesh point on the manifold; see Sec. 8.3; and (3) the function `manifplot` plots the results; see Sec. 8.2. The user may choose to use any of these functions independently, or all of them together.

To get a better grasp of how these steps work, Sec. 3 provides a walk-through of a sample system using the three-dimensional Hénon-like map defined in Sec. 2.

5 Step 1: Initialising the system (`StdHenon3D`)

All the functions (methods) that need to be provided by the user to define the system are consolidated into a single class, which we refer to as `StdHenon3D` in this guide; this class name can be chosen arbitrarily and `StdHenon3D` is simply used as an example.

Typically, the user is required to provide a few essential methods in this class, including the definition of the diffeomorphism f (`ff`), its inverse f^{-1} (`ff_inv`), and the fixed point associated with the manifold to be computed (`fixpoints`). In addition to the primary fixed point, the user has the flexibility to include any number of fixed points of the map for future reference. Additionally, this class allows the user to specify a phase space compactification as `compactify` and `decompactify`. If no compactification is required for the system, the corresponding methods may be left empty.

An explanation of these functions is provided below. For a guide to the format and semantics of `StdHenon3D`, please, also refer to Table 1.

```
function map_points = ff(points, opts)
function map_points = ff_inv(points, opts)
function fixpinfo = fixpoints(points)
function comp_points = compactify(points)
function decomp_points = decompactify(points)
function outpoints = mapping(inpoints, opts, stab)
```

Methods: `ff(points, opts)` and `ff_inv(points, opts)`

These methods defines the diffeomorphism f and the inverse f^{-1} used in the computation. They are called by the method `mapping` to compute the images (preimages) of points on the unstable (stable) manifold.

These functions have two arguments:

points: a structure containing the fields `x`, `y` and `z` that are the x -, y - and z -coordinates of the points on which to apply the function f or f^{-1} .

methods		
name	user-provided	content
ff	yes	diffeomorphism f
ff_inv	yes	diffeomorphism f^{-1}
fixpoints	yes	fixed points
compactify	yes	compactification of the space
decompactify	yes	inverse of the compactification
mapping	no	applies f or f^{-1}

Table 1: Summary of the class of methods passed to the algorithm.

opts: user-defined options, as explained in Sec. 6; only the field **par** from **opts** is passed to **ff** or **ff_inv**, which contains the parameter values needed for the computation.

and they return the structure:

map_points: contains the fields **x**, **y** and **z** representing the x -, y - and z -coordinates of the (pre)images of the input structure **points**.

Method: **fixpoints(points)**

This method returns the fixed points of the system and contains their explicit formulae. It is mandatory to include at least the fixed point associated with the manifold to be computed, but additional fixed points of the system can be added for reference.

This method has one argument:

opts: user-defined options, as explained in Sec. 6; only the field **par** from **opts** is passed to **fixpoints**, which contains the parameter values needed for the computation.

and it returns the structure:

fixpinfo: explicit coordinates of the fixed point for the chosen parameter values.

Method: **compactify(points)** and **decompactify(points)**

For our example map **StdHenon3D**, we use compactified coordinates. Hence, we introduce two functions **compactify** and **decompactify** that define the coordinate transformations of the compactification and its inverse, respectively. They are called by the method **mapping**. The user needs to change them to the correct transformation, or set them as the identity if they work in noncompactified coordinates.

Each method has one argument:

points: a structure containing the fields **x**, **y** and **z**, which are the x -, y - and z -coordinates of the points to compactify or decompactify.

and it returns the structure:

- comp_points:** a structure containing the fields `x`, `y` and `z` of compactified coordinates x , y and z , or
- decomp_points:** a structure containing the fields `x`, `y` and `z` of decompactified coordinates x , y and z .

Method: `mapping(inpoints, opts, stab)`

The method `mapping` is used by the main routine `GrowFundCurv1D` to compute the (pre)images of a set of points. Typically, the user does not need to modify this method. It contains three arguments:

- inpoints:** a structure containing the fields `x`, `y` and `z`, which are the x -, y - and z -coordinates of the points to which the map is applied.
- opts:** user-defined options, as explained in Sec. 6; there are two fields from `opts` passed to `mapping`. Namely, the field `thesystem`, which contains the class for correctly calling other methods, and the field `par` consisting of a structure with the parameter values.
- stab:** an integer which refers to the stability of the manifold to be computed; 1 for stable manifolds and -1 to unstable manifolds. This information is not set by the user; it is set during the execution of the main routine `GrowFundCurv1D`.

It applies the following three submethods:

- the method `decompactify`, which applies the inverse of the compactification to the structure of points to obtain the original coordinates;
either `ff` or `ff_inv`, based on the stability of the manifold; and
- the method `compactify`, to return to compactified coordinates.

and it returns the structure:

- outpoints:** contains the fields `x`, `y` and `z` representing the x , y and z coordinates of the (pre)image of the input structure `inpoints`

6 Step 2: Definition of parameters and options

All the information needed for the computation of the manifold is stored by the user in a structure named `opts`, as shown in the demo script. Table 2 shows a summary of all the fields of `opts`.

`opts.thesystem = StdHenon3D;`

specifies the user-defined class that has the diffeomorphism and fixed points. This structure serves the purpose of identifying the appropriate class when the manifold computation is in progress.

opts			
field	subfields	content	meaning
thesystem		class	Name of the class where the system is defined
par	{par1, par2, ...}	$\mathbb{R}$	Values (and names) of the parameters
target_arc		$\mathbb{R}$	Approximate arclength of the manifold
max_funditer		$\mathbb{N}$	Maximum number of iterations of the fundamental domain
name_fixpoint		string	Name of the fixed point associated to the manifold. It has to match its name in the class
stability		string	Stability of the 1D manifold: Unstable: 'Umanifold' Stable: 'Smanifold'
branch (optional)		string	Which branch to compute: 'pos' or 'neg'
accpar (optional)	{alphamax, deltalphamax, deltamin, deltamax, init_step}	$\mathbb{R}$	Accuracy parameters. Default: {0.3, 10^{-3} , 10^{-6} , 10^{-2} , 10^{-7} }

Table 2: Options defined in the structure **opts** that initialise the setting needed to compute the (un)stable manifold.

opts.par = struct('a', 4.2, 'b', -0.3, 'xi', 0.8);

sets the names and values of the parameters of the system. The names of the parameters have to be consistent with the ones defined in the system class **StdHenon3D** from Sec. 5.

opts.target_arc = 2000;

sets the target arclength of the manifold computation to 2000. During the iterative process, the algorithm monitors the cumulative arclength of the manifold and only computes the mesh points necessary to reach this specified value.

opts.max_funditer = 100;

sets the maximum number of iterations of the fundamental domain to 100. If the iteration count reaches this limit, the algorithm terminates the manifold computation, even if the target arclength **opts.target_arc** has not yet been achieved.

The options **opts.target_arc** and **opts.max_funditer** serve as the primary stopping criteria for the algorithm. The optimal values for these parameters are highly system-dependent. We recomend to start with low values and increase them as needed to achieve the desired manifold arclength.

opts.name_fixpoint = 'pplu';

identifies the name of the fixed point for which the manifold will be computed; in our case, the fixed point p^+ . This information is passed to the main routine, `GrowFundCurv1D`, which uses it to initialise the manifold data. The string assigned here must exactly match one of the names defined in the `fixpoints` method of the system class from Sec. 5.

`opts.stability = 'Umanifold';`

a string that specifies that the algorithm will compute an unstable manifold. The alternative option is `'Smanifold'`, which indicates the computation of a stable manifold.

`opts.branch = 'pos';`

it identifies the particular branch of the computed (orientation-preserving) manifold—here, `'pos'` or `'neg'` refers to the branch that lies in the direction to the right or left of the x -coordinate of the fixed point, respectively. If the manifold is non-oriented, this field only determines the direction in which the first fundamental domain is computed; however, both branches are computed regardless. If this field is omitted, the algorithm defaults to the sign of the initial step δ from `opts.accpar.init_step`, defined later in this section.

For oriented manifolds, if the user wishes to include the other branch to `manif` after executing the main routine `GrowFundCurv1D`, they can use the function:

`manif = add_branch(manif, opts, 'neg');`

`opts.accpar`

contains fields that store the parameters that control the accuracy of the computations. These fields, default values and parameters are:

`alphamax = 0.3;` the maximum $\alpha_{\max}$ of the angle allowed between triplets of mesh points,

`deltalphamax = 10-3;` the maximum curvature $(\Delta\alpha)_{\max}$ allowed between triplets of mesh points,

`deltamin = 10-6;` the minimum $\Delta_{\min}$ of the permitted distances between mesh points.

`deltamax = 10-2;` the maximum $\Delta_{\max}$ of the permitted distances between mesh points.

`init_step = 10-7;` the initial step size δ from the fixed point in the direction of the eigenvector (for defining the first fundamental domain).

Setting these parameters is optional and their default values are chosen to ensure a good balance between computational efficiency and accuracy for a wide range of systems. However, users can adjust these parameters based on the specific characteristics of their dynamical system and the desired precision of the manifold computation. For a comprehensive understanding of these accuracy constraints, please consult the referenced source [2]. For an example of how the first fundamental domain is computed

using the initial step δ from `opts.accpar.init_step`, we refer to [1].

7 Step 3: the main routine `GrowFundCurv1D` and structure `manif`

The core of the package starts with the user-provided options `opts`, as described in Sec. 6. This structure is passed as the sole argument to the main routine `GrowFundCurv1D`. During execution, the function `init_manif`, which is typically not modified by the user, is called to initialise the `manif` structure, which stores all the information of the manifold computation data. The details of the `manif` structure are explained in Sec. 7.1.

`manif = GrowFundCurv1D(opts);`

The main routine `GrowFundCurv1D` computes the manifold associated with a fixed point by successively iterating a fundamental domain. While previous versions required a user-provided initial fundamental domain, version 3.0 and later automates this process. The algorithm now computes the initial fundamental domain internally in the direction of the initial distance δ , defined by `opts.accpar.init_step`, or alternatively via the `opts.branch` setting; see Sec. 6.

For orientation-reversing manifolds, the algorithm computes both branches of the manifold, starting from the automatically defined fundamental domain which ‘flips’ across the fixed point. For orientation-preserving manifolds, it computes a single branch corresponding to the initial fundamental domain. If a user wishes to include the second branch for an orientation-preserving manifold after the initial run, they can use the `add_branch` function:

`manif = add_branch(manif, opts, ‘neg’);` for adding the negative branch, or
`manif = add_branch(manif, opts, ‘pos’);` for adding the positive branch.

Note that the `add_branch` function may take a different `opts` structure if the user wants to modify any of the options for the new branch computation.

During the iterative process of the fundamental domain, version 3.0 and later also stores the index of the approximate preimage for each mesh point, enabling the construction of pseudo-orbits from any point on the manifold. A comprehensive explanation of the algorithm is available in [2], and specific details regarding the first fundamental domain and pseudo-orbit computation provided in [1].

7.1 `manif`: Structure with the manifold data

This structure is the output of the main routine `GrowFundCurv1D`. It contains all information regarding the system, the manifold computation, and the accuracy settings used. Table 3 provides a summary of all fields in `manif`, and the following subsections explain each of them in detail.

manif.name = 'Wu_pplu' – Manifold Name

This field contains a descriptive string used to identify the manifold. It is generated by the `init_manif` function during the execution of the main routine `GrowFundCurv1D`. The name is constructed by concatenating two strings:

- (1) `'Wu_'` or `'Ws_'`; a prefix indicating an unstable or stable manifolds; respectively. This is generated with the information from `opts.stability`.
- (2) `'pplu'`; the user-defined name of the associated fixed or periodic point, taken from `opts.name_fixpoint`. In the provided demo, this corresponds to the fixed point p^+ of the system.

manif.orientability = 'orientation-reversing' – Orientation Properties

This field is a descriptive string that indicates the orientation property of the manifold. It is generated by the `init_manif` function during the execution of the main routine `GrowFundCurv1D`. The value assigned to `manif.orientability` is based on the associated eigenvalue of the fixed point. If the eigenvalue is positive, the field is assigned the string `'orientation-preserving'`; if the eigenvalue is negative, it is assigned the string `'orientation-reversing'`.

manif.stability = 'Umanifold' – Stability of the Manifold

This field is a string that specifies the stability of the manifold to be computed. It is generated by the `init_manif` function during the execution of the main routine `GrowFundCurv1D`. The value of `manif.stability` is derived from the user-defined `opts.stability` field, taking the value `'Umanifold'` for unstable manifolds or `'Smanifold'` for stable manifolds.

manif.dimension = 1 – Dimension of the Manifold

This field is an integer representing the dimension of the manifold to be computed. It is generated by the `init_manif` function during the execution of the main routine `GrowFundCurv1D`. For unstable manifolds, this value corresponds to the number of associated eigenvalues with a modulus greater than one ($|\lambda| > 1$); for stable manifolds, it corresponds to the number of eigenvalues with a modulus less than one ($|\lambda| < 1$).

Please note that this algorithm is specifically designed to compute one-dimensional manifolds. If the eigenvalues indicate a two-dimensional manifold, the algorithm issues a warning. However, the user may still proceed; in such cases, the algorithm computes a one-dimensional branch along a specified direction in the 2D local subspace.

manif.fixp – Fixed Point Information

This field is a structure containing the information of the fixed point associated with the manifold computation. If a phase space compactification has been specified, the coordinates and eigenvectors provided here are expressed in those compactified coordinates. This structure is automatically generated by `init_manif` during the execution of `GrowFundCurv1D` and contains the following subfields:

name: the user-provided name for the fixed point.

`x, y, z`: the coordinates of the fixed point.

`eigval`: the eigenvalues associated with the fixed point.

`eigvec`: the corresponding eigenvectors, where each column represents the eigenvector for the first, second, and third eigenvalue, respectively.

`Smanifold`: a structure containing the information for the stable manifold associated with the fixed point. It includes the dimension, orientability properties, and the associated eigenvalues and eigenvectors for the stable subspace.

`Umanifold`: a structure containing the information for the unstable manifold associated with the fixed point. It includes the dimension, orientability properties, and the associated eigenvalues and eigenvectors for the unstable subspace.

manif.system_info – System Information

This field is a structure containing the core information of the dynamical system. It is automatically generated by the `init_manif` function during the execution of the main routine `GrowFundCurv1D` and contains the following subfields:

`thesystem`: the class of methods defined by the user that stores the system's functions, fixed points and compactification.

`par`: the user-provided parameter names and values used for the computation.

`fixp`: information for all fixed points defined by the user in the `fixpoints` method of the system class. This subfield follows the same format as `manif.fixp` and contains the coordinates, eigenvalues, eigenvectors, stability properties, and manifold information for each point.

manif.grow_info – Computation Information

This field is a structure containing the manifold computation information and the accuracy settings used. Most of its fields are automatically generated by `init_manif` based on `opts`, `manif.system_info`, or default parameters, all of those already defined in previous sections. These include the fields and values for:

`alphamax`, `deltalphamax`, `deltamin`, `deltamax`, `init_step`, `thesystem`, `stability`, `orientability`, `dimension`, `eigval`, `eigvec`, `max_funditer`, and `target_arc`.

Its last field is `runinfo`, which is written by the main routine `GrowFundCurv1D` and stores performance statistics and execution data. The subfields of `runinfo` are:

`rem_deltamin`: number of points removed during the execution of `GrowFundCurv1D` due to mesh points lying too close together ($< \Delta_{\min}$).

`rem_nan`: number of points removed during the execution of `GrowFundCurv1D` due to returning NaN.

`rem_inf`: number of points removed during the execution of `GrowFundCurv1D` due to returning INF from being mapped to infinity via compactification.

add_alphamax: number of points added by triggering the accuracy condition $\alpha_{\max}$.
add_deltamax: number of points added by triggering the accuracy condition $\Delta_{\max}$.
add_deltalphamax: number of points added by triggering the accuracy condition $(\Delta\alpha)_{\max}$.
num_funditer: number of fundamental domain iterations performed in this run.
total_arclength: total arclength of the manifold, including all branches computed in this run.
return_distance: the distance η from the first iteration. It issues a warning if greater than the initial distance δ defined in `opts.init_dist`. For a detailed definition, see [1].
time: total CPU time spent in this run.

manif.points – Mesh Point Data

This field is a structure containing the data and mesh points generated during the manifold computation. It is generated by the main routine `GrowFundCurv1D` and contains subfields for each computed branch: `'pos'` for the positive branch and/or `'neg'` for the negative branch. Each branch subfield contains the following data:

x, y, z: the x -, y -, and z -coordinates of the mesh points.
arc: the cumulative arclength along the manifold for each mesh point.
fund_idx: an $M \times 2$ matrix storing the indices for the M computed fundamental domains. Each row represents a single fundamental domain, with the first and second columns containing its starting and ending mesh point indices, respectively.
pre_idx: the index of the approximate preimage for each mesh point. This enables the construction of pseudo-orbits using the function `pseudo_orbit`, as described in Sec. 8.3.
pre_branch: identifies the branch (`'pos'` or `'neg'`) from which each approximate preimage originates. Also used by the function `pseudo_orbit` for the pseudo-orbit construction.

8 Step 4: Post-processing the Results

Once the manifold has been computed, the final step is to process the data for analysis and visualisation. We have implemented specialised functions for this purpose: `inter_plane` determines the ordered intersection points of the manifold with a pre-specified plane Σ ; the function `pseudo_orbit` reconstructs pseudo-orbits starting from any mesh point on the manifold; and `manifplot` plots the results.

manif			
field	subfields	type	definition
name		string	Name of the manifold
orientability		string	Orientation properties of the manifold (' orientation-preserving ' or ' orientation-reversing ')
stability		string	Stability of the manifold (' Umanifold ' or ' Smanifold ')
dimension		$\mathbb{N}$	Dimension of the manifold
fixp	See Table 4	divers	Information of the associated fixed point
system_info	{par, fixp}	$\mathbb{R}$, divers	Parameter names, values and the fixed points information (same style as manif.fixp structure)
grow_info	See Table 5	divers	Manifold computation information and settings
points	{x, y, z, arc, fund_iter, pre_idx, pre_branch}	$\mathbb{R}$, $\mathbb{N}$, string	Coordinates, arclength and approximate preimage of mesh points

Table 3: Names and definitions of the fields and subfields in the structure **manif**.

manif.fixp		
subfield	type	definition
name	string	Name of the fixed point
x, y, z	$\mathbb{R}$	Coordinates of the fixed point
eigval	$M_{1,3}(\mathbb{R})$	Eigenvalues associated to the fixed point
eigvec	$M_{3,3}(\mathbb{R})$	Eigenvalues associated to eigvec .
Smanifold	struct	Information of the stable manifold
Umanifold	struct	Information of the unstable manifold

Table 4: Names and definitions of the subfield **manif.fixp**, where $M_{n,m}(\mathbb{R})$ is an $m \times n$ matrix with elements in $\mathbb{R}$

8.1 Function **manif = inter_plane(manif, angle)**

This function computes the (ordered) intersection points of the manifold with a pre-specified plane Σ . For the specific example provided in this manual, we use a half-plane perpendicular to the (x, y) -plane passing through the origin. This plane is defined by an **angle** $\theta \in [-\pi, \pi]$. It requires two input arguments:

manif: the structure containing the manifold computation data generated by **GrowFundCurv1D**.

manif.grow_info		
subfield	type	definition
alphamax	$\mathbb{R}$	Maximum $\alpha_{\max}$ of the angle allowed between triplets of points
deltalphamax	$\mathbb{R}$	Maximum curvature $(\Delta\alpha)_{\max}$ allowed between triplets
deltamin	$\mathbb{R}$	Minimum $\Delta_{\min}$ of permitted distance between mesh points
deltamax	$\mathbb{R}$	Maximum $\Delta_{\max}$ of permitted distance between mesh points
init_step	$\mathbb{R}$	Initial step δ for initial fundamental domain
thesystem	class	User-defined class of methods. In our demo: <code>StdHenon3D</code>
stability	string	Stability of the manifold
orientability	string	Orientation properties of the manifold
dimension	$\mathbb{N}$	Dimension of the manifold
eigval	$\mathbb{R}$	Eigenvalue used in the computation
eigvec	$\mathbb{R}^3$	Eigenvector used in the computation
max_funditer	$\mathbb{N}$	Maximum number of iterations of the fundamental domain
target_arc	$\mathbb{R}$	Target arclength for the manifold computation
runinfo	struct	Performance statistics and execution data; see Table 6

Table 5: Names and definitions of the subfield `manif.growinf`. The majority of the fields are initialised in the `opts` structure from Sec. 6

manif.grow_info.runinfo		
subfield	type	definition
rem_deltamin	$\mathbb{N}$	Number of points removed due to being with distance $\Delta_{\min}$
rem_nan	$\mathbb{N}$	Number of points removed due to presence of NaN
rem_inf	$\mathbb{N}$	Number of points removed from infinity (compactification)
add_alphamax	$\mathbb{N}$	Number of points added because $\alpha_{\max}$ is too large
add_deltamax	$\mathbb{N}$	Number of points added because $\Delta_{\max}$ is too large
add_deltalphamax	$\mathbb{N}$	Number of points added because $(\Delta\alpha)_{\max}$ is too large
num_funditer	$\mathbb{N}$	Number of fundamental domain iterations
total_arclength	$\mathbb{R}$	Total arclength including all branches
return_distance	$\mathbb{R}$	Distance η from first iteration
time	$\mathbb{R}$	Total CPU time spent in this run

Table 6: Names and definitions of the subfield `manif.growinf.runinfo`

angle: a string representing the angle $\theta \in [-\pi, \pi]$ of the half-plane that the manifold intersects. This angle is defined relative to the x -axis and is measured counter-clockwise.

In the provided demo, we use the plane Σ defined by $\theta = \pi/4$, which corresponds to

manif.inter	
field	meaning
angle	The angle of the half-plane with respect to the x -axis
plane	Name of the half-plane used
points.('branch').x	The x -coordinate of the intersection points of the branch ('branch')
points.('branch').y	The y -coordinate of the intersection points of the branch ('branch')
points.('branch').z	The z -coordinate of the intersection points of the branch ('branch')
points.('branch').idx	The indices of the corresponding mesh point of the branch ('branch')

Table 7: Name and meaning of fields of **manif.inter**

the half-plane:

$$\Sigma = \{(x, y, z) \in \mathbb{R}^3 \mid x = y, x > 0\}.$$

The intersection points are computed by detecting sign changes with respect to a normal vector of the plane Σ . The precise intersection coordinates are then determined using cubic spline interpolation from nearby points. Each interpolated intersection point is stored in the subfield **inter.points** of the structure **manif**. The ordering of the intersection points is induced by the fact that the manifold is parameterised by arclength. Although the plane used in our sample map **StdHenon3D** is defined in specific terms, the user may change the definition of the plane in the function **inter_plane**.

The output of the function **inter_plane** is stored in the field **inter** that is added to the structure **manif**; its subfields are detailed below and summarised in Table 7.

manif.inter – Intersection Points

This field is appended to the **manif** structure during the execution of the **inter_plane** function. It contains the data of the intersection points of the manifold with a planar section. It includes the following subfields:

- angle:** a string representing the angle $\theta \in [-\pi, \pi]$ of the half-plane that the manifold intersects. This angle is defined relative to the x -axis and is measured counter-clockwise.
- plane:** a string representing the name of the half plane.
- points.(branch):** a structure containing the intersection data for each computed branch. Here, 'branch' is 'pos' and/or 'neg'.

The subfields of **points.(branch)** are:

- x, y, z:** the x -, y -, and z -coordinates of the intersection points, determined via cubic spline interpolation between mesh points.
- idx:** the index of the mesh point immediately preceding the intersection point. This ordering is induced by the fact that the manifold is parameterised by arclength.

8.2 Function `h = manifplot(manif);`

This function plots the results obtained from `GrowFundCurv1D`. It takes the structure `manif` as input, which contains all the relevant information about the computed manifold. It returns a handle `h` to the generated plot. If the function `inter_plane` has been called and `manif` has the field `inter`, then the plot also includes the intersection points, and the half plane Σ .

Note that this function automatically plots stable manifolds in a blue gradient, and unstable manifolds with a red gradient. To improve the three-dimensional visualisation of the data, we use the `color_line3` function by [3]. This routine transforms the one-dimensional manifold into a surface object, making it possible to apply a colormap to the plot. In our specific implementation, the colormap is defined based on the distance from each point to the vertical line $(x, y) = (0, 0)$. This generates an illusion of light and depth, that improves the visual representation of the manifold's geometry. An example of the output generated by `manifplot` is shown in Fig. 1.

8.3 Function `orbit = pseudo_orbit(manif, idxpoint, branch);`

This function traces back a pseudo-orbit ending at any pre-specified mesh point in the manifold. The input arguments for `pseudo_orbit` are:

- `manif`: the structure containing the manifold computation data generated by `GrowFundCurv1D`.
- `idxpoint`: an integer representing the index of a mesh point in the structure `manif.points`, which is the last point in the pseudo-orbit.
- `branch`: a string ('pos' or 'neg') identifying the branch where the selected mesh point is located.

The pseudo-orbit is reconstructed using the approximate preimage information stored in the `pre_idx` and `pre_branch` fields of `manif.points`. The resulting `orbit` structure contains the following fields:

- `name`: a string array indicating the branch origin for each point in the pseudo-orbit.
- `x, y, z`: the x -, y -, and z -coordinates of the orbit points, extracted directly from the mesh points of the manifold.
- `idx`: the index of the mesh point that corresponds to each point in the pseudo-orbit.

In the provided demo, we compute a pseudo-orbit whose endpoint corresponds to the (approximate) final computed intersection point of the positive unstable manifold branch $W^u(p^+)$ with the half-plane Σ . In Fig. 1, this orbit is visualised as empty circles connected by dotted lines. This figure was made by overlaying the pseudo-orbit on the main manifold plot generated by `manifplot`.

References

- [1] D. C'Julio, S. Amani, S. Doak, B. Krauskopf, and H. Osinga. Finding pseudo orbits on invariant manifolds of maps as seeds for boundary value problems. .
- [2] D. C'Julio, B. Krauskopf, and H. Osinga. Computing parameterised large intersection sets of 1D invariant manifolds: a tool for blender detection. .
- [3] G. Stillfried. 3d colored line plot, 2022. MATLAB Central File Exchange. <https://www.mathworks.com/matlabcentral/fileexchange/23566-3d-colored-line-plot>.